

CT500 FirmWare User Guide

Version 0.2

Shmt Co., Ltd.

REVISION HISTORY PAGE

Issue	Rev No.	Originator	Details of Change	Date
1	0.1	DJKIM	Initial Version	2007-02-14
2	0.2	DJKIM	Add HOWTO porting	2007-02-23
3				
4				
5				

Confidential

Technical documents for Firmware

Copyright © 2007 Shmt Limited. All rights reserved.

Confidentiality Status

This document is **NOT OPEN ACCESS**. The distribution is restricted to certified company.
This document is **CONFIDENTIAL**.

Feedback on this document

If you have any comments on this document, please contact your channel to our company.

Shmt Web address

<http://www.shmt.com>

Confidential

Contents

1. INTRODUCTION	7
2. OverView	8
3. SW Hierarchy	12
4. Directory Analysis	13
4.1. main	16
4.1.1. app	16
4.1.2. bootloader	16
4.1.3. startup	16
4.2. sys	17
4.3. peri	17
4.4. inc	18
4.5. lib	18
4.5.1. api	19
4.5.2. drivers	19
4.5.3. basic peripheral	19
5. HOWTO build execute file	20
5.1. Linux Base	20
5.1.1. makefile modification	21
5.2. ADS v1.2 Using	21
5.2.1. FirmWare Project Structure	21
6. HOWTO Porting	23
6.1. Memory map	23
6.2. System setting	23
6.3. Source modification	23
APPENDIX	24
A. ASSEMBLER DIRECTIVES	24
C. CACHE AND MMU INSTRUCTION	26

Figures

[Fig - 1] FirmWare S/W Hierarchy	12
[Fig - 2] Pre-level driver directory tree.....	13
[Fig - 3] F/W directory tree	16
[Fig - 4] Cygwin상에서 makefile 실행	20
[Fig - 5] ADSv1.2 Projcet Structure.....	22
[Fig - 6] 도메인 액세스 컨트롤 레지스터 cp15:c3 의 포맷	26
[Fig - 7] MMU 안에 위치한 cp15:c1 레지스터 컨트롤 비트	26
[Fig - 8] 고속 문맥 전환 레지스터 cp15 레지스터 13	27
[Fig - 9] 변환 테이블 베이스 주소 cp15 레지스터	27
[Fig - 10] Way와 세트 인덱스 주소 지정 방식을 사용한 cache clean시 cp15:c7 레지스터 Rd의 형식	28
[Fig - 11] 주메모리 안에서 원래 값으로 참조된 cache line을 clean하고 flush할 때의 cp15:c7 레지스터 포맷.....	28
[Fig - 12] Lock down bit 사용 cache lock했을 때 cp15:c9 레지스터 포맷	29

Tables

[Table - 1] H/W 수정 및 추가 시 **FirmWare** 소스 수정 사항오류! 책갈피가 정의되어 있지 않습니다.

[Table - 2] TLB 플러시하기 위한 **cp15:c8** 명령어..... 26

[Table - 3] TLB 락다운 레지스터를 액세스하는 명령 26

[Table - 4] MMU 제어 컨트롤 레지스터 **cp15:c1** 의 비트필드 27

[Table - 5] **Cache flush** 명령어 27

[Table - 6] **Way**와 세트 인덱스 주소 지정 방식을 사용한 **cache line clean & flush cp15:c7** 명령어 27

[Table - 7] 주메모리 안에서 위치를 참조하여 **cache line clean**하고 **flush**하는 명령어 28

[Table - 8] **Lock down bit**를 사용하여 **cache**를 **lock**하기 위한 **cp15:c9** 명령어..... 28

1. INTRODUCTION

본 문서는 가라오케 시스템을 위한 low-cost solution SOC로 개발된 CT-500의 firmware에 대한 내용을 다룬다. H/W 설계 과정 및 FPGA 검증 과정에서 사용되는 firmware의 내용과 디렉토리 및 S/W hierarchy가 설명되었다.

Confidential

2. OverView

CT500 은 ARM926EJ-S을 내장하고 있으며 다양한 어플리케이션에 효율적으로 대응할 수 있는 SOC 칩으로 다음과 같은 features를 갖는다.

- ARM926EJ core
 - Normal operation mode : 100MHz
 - 2x operation mode : 200MHz
- Video Encoder
 - NTSC/PAL display
 - One DAC for CVBS, two times over-sampled with 10 bit resolution.
 - Internal Color Bar Generator.
 - Cross color reduction filter.
- Internal PSRAM
 - 4 MBits
 - Optional configuration
- UART 4 Channel
 - UART 32 depth FIFO 내장한 버전 2 ch
 - UART 8 depth FIFO 내장한 버전 2 ch
- Graphic Engine
 - NTSC/PAL output mode : 704 x 480i / 704 x 576i (max)
 - VGA output mode : 640 x 480p(VGA), 320x240p(QVGA)
 - 4pages
 - Wide(1440x576) x 2page : no Palette
 - VGA (640x576) x 2page : Palette 256 color/page
 - Overlay & α-blending : 3pages(Wide 1 page + VGA 2pages).
 - Support chroma keying : Color selectable.
 - Line Up-scaling : 480 -> 576 (NTSC -> PAL).
 - Up-scaling support function
 - For helping NTSC->PAL conversion.
 - Insert 1 horizontal line, copied from the 5th line, each 5 lines.

No line filter required.

Configurable Outputs : RGB(565) & Sync/ITU-R BT.601(16bit)/ITU-R BT.656(8bit).

Internal Color Bar Generator.

Color Space Conversion.

■ DDR Controller

128/256/512 Mbit

■ I2C

Variable clock rate : 400kHz max.

■ I2S

Configurable Master/Slave.

ADC(receive) & DAC(transmit) IF for audio Codec.

IF mode : Standard I₂S, MSB(=Left) justified mode(configurable).

8/16/24/32bit Word lengths.

16k/32k/44.1k/48kHz Sampling Frequency.

Each 16 word FIFO entries for Transmit & Receive.

■ Timer

■ Enhanced-DMA

■ Graphic DMA

■ SPI

SPI clock speed up to 12MHz.

Ch-1 : For transferring MP3 bit-stream.

160Byte/10mS @128k Sampling.

Support DMA.

Ch-2 : For transferring commands.

Data Length(max) : 64bit(8byte).

No needs DMA.

■ UART

Special UART (2ch)

Baudrate : 115200 max

Tx 32bytes & Rx 32bytes FIFOs.

31.25k baudrate support for MIDI.

Standard UART (2ch)

Baudrate : 115200 max

Tx 8bytes & Rx 8bytes FIFOs.

- Multi-boot

For support wide range application

- Sound Engine

Mask rom for Sound Engine : 32 / 64 / 128 MBits

Internal SRAM for Sound Engine : 1 MBits + 24K bits

Operation Clock : 100MHz.

(128K + 3072)Byte SRAM

Sound effect & MIC echo memory.

512Byte ROM.

Audio CODEC : Excluded.

Wave Playback & Recorder Interface.

1024 point FFT with I2S receiver.

4ch I2S Master interface : 2ch ADC in & 2ch DAC out .

PCM Rx/Tx interface for Record & Play.

- NAND Flash memory controller

32M ~ 2G Byte.

512/256 byte, 1-bit ECC.

Command FIFO/Data FIFO.

Support small/large block.

Support SLC/MLC.

- USB 2.0

USB 1.1/2.0 compliant.

PHY included.

- External Video Input

Digital Video Decoder Interface : ITU-R BT.656(8bit).

Supports Interlace / Non-interlace Mode.

Color Space Conversion.

Memory Buffering.

Support Frame Capture.

- Extra Asynchronous Chip select signal for reservation : 4 (8/16bit).

Async IF – ex) ROM/Nor-flash/SRAM IF.

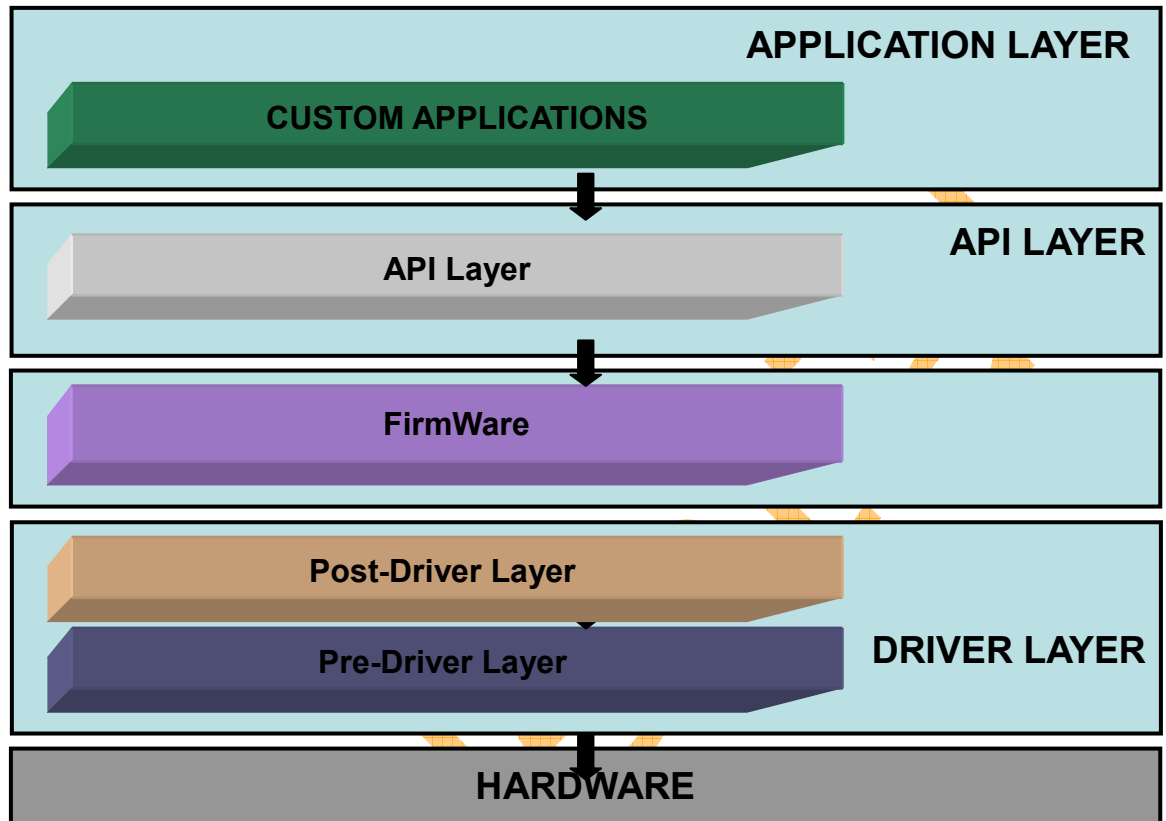
Wait configurable.

Upto 128M Byte.

Confidential

3. SW Hierarchy

다음 그림은 CT-500 의 firmware S/W hierarchy를 나타낸다.



[Fig - 1] FirmWare S/W Hierarchy

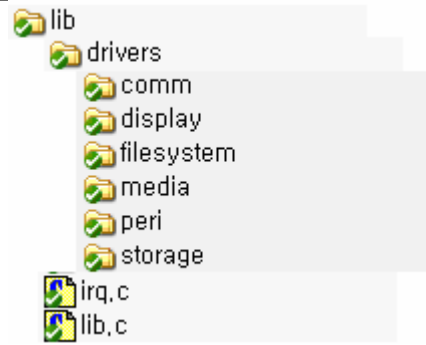
CT-500 의 H/W를 firmware로 구성함에 있어 S/W firmware layer는 세가지로 구분하였다.

1) Pre-level driver layer

기본 peri(Memory, Timer/PWM, WDT, GPIO, VIC, ADC, Power management etc.)와 통신(UART, I2C, I2S, SPI etc), 디스플레이(LCD, Video Encoder), 미디어(AUDIO), 스토리지(NAND)의 register level 동작을 조작한다.

기본 peri에 대한 pre-level driver 함수는 lib/, lib/drivers/peri/ 디렉토리에 peri 모듈 별 파일로 구성이 되어 있으며, 함수 명은 `smtfuncName()`로 지정된다.

통신, 디스플레이, 미디어, 스토리지의 모듈 별 파일 구성도 이와 비슷하다. Pre-level driver 함수들은 `module_pre_drv.c` 파일로 존재하며 다음은 pre-level driver 함수의 디렉토리 구조를 나타낸다.



[Fig - 2] Pre-level driver directory tree

2) Post-level drivers layer

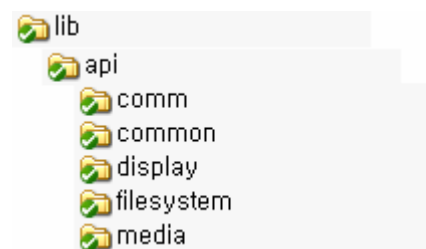
Pre-level driver layer보다 상위 driver layer로 pre-driver function을 호출하여 scenario적인 관점의 동작을 조작한다. Pre-driver 함수의 경우 register의 셋팅과 상태 체크 등 레지스터 레벨의 함수인 반면 post-driver 함수는 pre-driver 함수를 사용하여 read/write 등과 같은 scenario 동작과 관련된다.

Post-level driver 함수의 경우 pre-level driver 함수와 달리 함수명이 smt2로 시작한다. Post-level driver 함수들은 pre-level driver 함수들과 같은 디렉토리에 module_post_drv.c 파일로 존재한다.

3) API layer

API layer는 application 프로그램과 driver layer를 연결하는 계층으로 application 프로그램 개발자에게 디바이스에 대한 직접적인 접근이 필요 없도록 편리한 인터페이스를 제공한다.

API layer에는 file-system(FAT32), display(LCD), media(Audio) 관련 api 함수들이 있다.



[Fig - 3] API layer directory tree

4. Directory Analysis

상화에서 제공하는 F/W 프로그램의 디렉토리는 다음과 같다.

1) inc/ directory

Driver와 API 함수들의 선언과 사용되는 매크로, 전역 변수들을 담은 헤더 파일들이 있다.

api/

F/W api를 위한 헤더파일이 존재 (구축 준비 중)

drivers/

comm./

통신과 관련된 디바이스들(I2C, I2S, SPI, UART, USB etc)의 pre/post driver 헤더 파일들이 있다.

display/

이미지 디스플레이 디바이스(LCD, Video Encoder)의 pre/post driver 헤더 파일들이 있다.

filesystem/

파일 시스템을 위한 FAT32, partition 관련 헤더 파일이 있다.

media/

미디어(AUDIO) 디바이스의 H/W, S/W pre/post driver 헤더 파일이 있다.

peri/

storage/

Storage (NAND, SD/MMC) 디바이스의 pre/post driver 헤더파일이 있다.

fw_drv_err/

Post-level driver 함수의 에러 리턴을 위한 enum 정의 헤더파일이 있다.

commonmacro.h

- CT-500 F/W에서 사용할 수 있는 일반적인 매크로를 정의

global.h

- CT-500 F/W에서 사용할 수 있는 전역 변수의 정의

irq.h

- VIC를 위한 매크로와 함수 선언을 포함

lib.h

- Basic peripheral의 매크로 정의와 함수 선언을 포함

structdef.h

- CT-500 F/W에서 사용되는 데이터 타입을 정의

2) lib/ directory

Peripheral의 driver와 API 함수들이 정의 되어 있다.

api/

CT-500 API를 위한 함수 정의 파일이 존재 (구축 준비 중)

drivers/

CT-500 에서 사용한 pre/post-level driver 함수들이 정의 되어 있다.

comm./

I2C, I2S, UART, USB의 pre/post-level driver 함수 정의

display/

LCD, Video Encoder의 pre/post-level driver 함수 정의

filesystem/

FAT32 파일 시스템 post-level driver 함수 정의

media/

AUDIO의 post-level driver 및 mp3/ogg post-level driver 함수 정의

peri/

storage/

NAND, SD/MMC의 pre/post-level driver 함수 정의

irq.c

VIC 관련 pre/post-level driver 함수 정의

lib.c

Basic peripheral의 pre/post-level driver 함수 정의

3) main/ directory

- app/

User의 F/W 소스 및 application 소스가 위치하는 디렉토리

- bootloader/

NAND/NOR booting을 지원하기 위한 boot-loader 소스가 존재 (구축 준비 중)

- startup/

FirmWare main 함수로 진입하기 위해 필요한 ARM926 의 startup code와 memory configuration을 포함한다. (ADSV1.2 사용 할 경우 - [5.2 ADS v1.2 Using](#) 참조)

- startup-net/

- main.c

F/W 코드의 메인 함수

- mmu.c

ARM926EJ-S의 mmu setting을 위한 함수

- regview_v2.c

디버거 장비를 통한 디버깅 시 CT-500 의 레지스터 맵을 watch 할 수 있도록 레지스터 구조체 정의

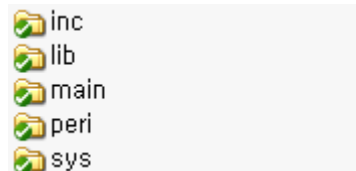
4) peri/ directory

각 peri 별 FPGA 테스트 파일과 헤더 파일이 존재한다.

5) sys/ directory

CT-500 의 system 레벨에서의 설정이 이루어진다. Memory 사이즈, clock 설정, boot mode 설정, memory bus width 등을 설정한다.

[Fig- 4]는 디렉토리 구조를 나타낸다.



[Fig - 4] F/W directory tree

Driver와 API 함수에 대한 자세한 사항은 programming guide를 참조하기 바람, 여기서는 스타트업 코드와 상화마이크로텍에서 제공하는 F/W 코드에서 공통적으로 사용하는 system configuration 관련 매크로 및 데이터 타입 정의에 대해서 설명하기로 한다.

4.1. main

4.1.1. app

CT-500 의 F/W 어플리케이션 소스가 있다.

4.1.2. bootloader

NAND/NOR booting에 사용되는 부트 로더가 있다.

4.1.3. startup

Startup 디렉토리의 파일들은 처음 reset 후 CT-500 의 동작 시 main() 함수로 진입하기 전 이루어지는 초기화 함수들이 포함된다.

다음은 startup directory에 있는 파일들에 대한 설명이다.

1) Startup.s

ARM926EJ core startup source file

2) smt926addr.inc

Startup.s에서 사용되는 memory bank configuration variable의 값을 각 bank별로 설정한다. 또한 bus, pll, clock configuration variable의 값을 설정한다.

3) asmlib.s

ARM926EJ-S core의 irq mode en/disable 어셈블리 함수가 있다.

4) mmufunc.s

ARM926EJ-S core의 MMU I/D-TLB의 clear assembly 함수가 있다.

4.2. sys

sys directory의 파일들은 CT-500 의 firmware 동작을 위한 system 관련 설정 헤더 파일들이다.

1) sysinc.h

Sys directory의 모든 헤더 파일을 포함한다.

2) sysdatadef.h

CT-500 의 firmware에서 사용되는 data type을 정의한다.

3) sysreg.h

CT-500 의 register map, 각 register 별로 할당된 비트 마스크를 정의한다.

4) syscfg.h

CT-500 의 boot mode, flash memory size 등 system에 관련 설정을 정의한다.

4.3. peri

peri 디렉토리의 파일들은 각 H/W 모듈 테스트 함수가 있는 파일들이다. 다음에서 설명하는 각 H/W 모듈들의 설명은 알파벳 순서에 따르도록 한다.

1) ADC

2) DMA

E-DMA(Enhanced)와 G-DMA(Graphic-DMA)로 구성되어 있으며, E-DMA는 memory to I/O, I/O to memory, memory to memory의 데이터 복사를 가능하게 한다.

G-DMA는 메모리에서 일정한 2 차원 영역을 다른 메모리 어드레스의 2 차원 영역으로 복사하는 일을 수행한다.

3) GPIO

4) PLL

5) Power Management

Power management 블록은 SOC 칩의 정상동작 시 normal 모드, 외부 클럭에 의한 slow 모드, CPU만 동작하는 idle 모드, CPU와 내부 로직이 동작하지 않는 sleep 모드를 지원한다. Idle 모드와 sleep 모드는 인터럽트에 의해 wake-up 할 수 있다.

6) TIMER & PWM

Timer는 interval, capture, match & overflow 모드를 지원한다.

- 7) WDT
- 8) VIC
- 9) Communication Modules
 - 가) I2C
 - 나) I2S
 - 다) SPI
 - 마) UART
 - 바) USB
- 10) Display Modules
 - 가) LCD
 - 나) Video Encoder
- 11) Media
 - 가) Audio
- 12) Storage
 - 가) Nand Flash
 - 나) SD/MMC
 - 다) SRAM
 - 라) PSRAM
 - 마) DDR-SDRAM

4.4. inc

CT-500 의 pre/post-level driver와 api에서 함수 구현을 위해 사용하는 헤더파일이 있다. API 함수들을 위해 api의 각 서브 디렉토리 별로 구성되어 있으며, driver 함수들을 위해서는 서브 디렉토리 별로 pre/post-level driver 헤더파일들이 있다. 또한 basic peripheral들을 위한 헤더파일(lib.h, irq.h)과 common 매크로 및 basic peripheral의 프로그램 구조체가 정의된 헤더파일(commonmacro.h, structdef.h)가 있다.

4.5. lib

CT-500 의 pre/post-level driver와 api 함수가 구현되어있다. API 각 서브 디렉토리 별로 해당 api 함수들이 구현되어 있으며, driver의 각 디렉토리 별로 pre/post-level driver 함수가, basic peripheral의 pre/post-level driver 함수는 irq.c, lib.c 파일에 구현되어 있다.

4.5.1. api

디바이스의 분류에 따라서 api는 comm.(I2C, I2S, UART, USB), display(LCD, Video Encoder), filesystem(NAND, SD/MMC), media(Audio, Video)로 구분된다. common(메모리/스트링)의 경우는 메모리 사용의 효율성을 위한 api로 디바이스의 분류와는 상관없다.

4.5.2. drivers

디바이스의 종류에 따라 comm.(I2C, I2S, UART, USB), display(LCD, Video Encoder), filesystem(fat32, partition), media(Audio, Video), storage(NAND, SD/MMC)로 구분되며 각 디바이스 별로 pre/post-level driver 함수들이 구현되어 있다.

4.5.3. basic peripheral

Timer, WDT, GPIO, VIC 등과 같은 기본적인 peri들의 pre/post-level driver 함수들은 irq.c, lib.c 파일에 구현되어 있다.

5. HOWTO build execute file

F/W의 실행 파일 생성은 windows와 linux OS 상에서 생성할 수 있도록 구성되어 있다. Windows에서는 ADSv1.2 IDE를 사용하며, linux에서는(Windows OS에서 는 cygwin 실행 후) makefile을 사용한다.

5.1. Linux Base

F/W 실행 파일 생성에 있어 linux-base 경우 makefile을 이용하여 생성한다. (Windows OS 경우 cygwin에서 makefile 실행)

Application 실행 파일의 생성은 F/W 프로그램의 root 디렉토리에서 다음과 같다.

1) 처음 실행 파일 생성 시

`make clean all`

* 기존의 모든 오브젝트를 삭제 후 모든 소스를 재 컴파일하여 실행 파일을 생성한다.

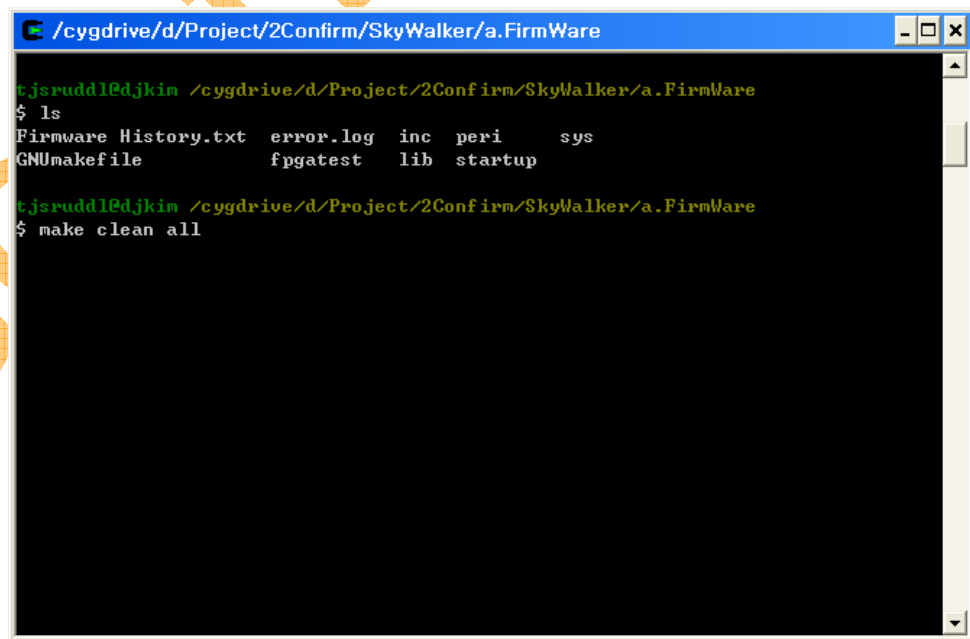
2) 소스 수정 후 실행 파일 생성 시

`make dep`

`make all`

* 수정된 소스의 **dependency**를 체크한 후 실행 파일을 생성한다.

[Fig-5] 는 windows상에서 cygwin을 통한 makefile의 실행을 보여준다.



```

/cygdrive/d/Project/2Confirm/SkyWalker/a.FirmWare
tjsruddl@djkin /cygdrive/d/Project/2Confirm/SkyWalker/a.FirmWare
$ ls
Firmware History.txt  error.log  inc  peri  sys
GNUmakefile          fpgatest  lib  startup
tjsruddl@djkin /cygdrive/d/Project/2Confirm/SkyWalker/a.FirmWare
$ make clean all
  
```

[Fig - 5] Cygwin 상에서 makefile 실행

5.1.1. makefile modification

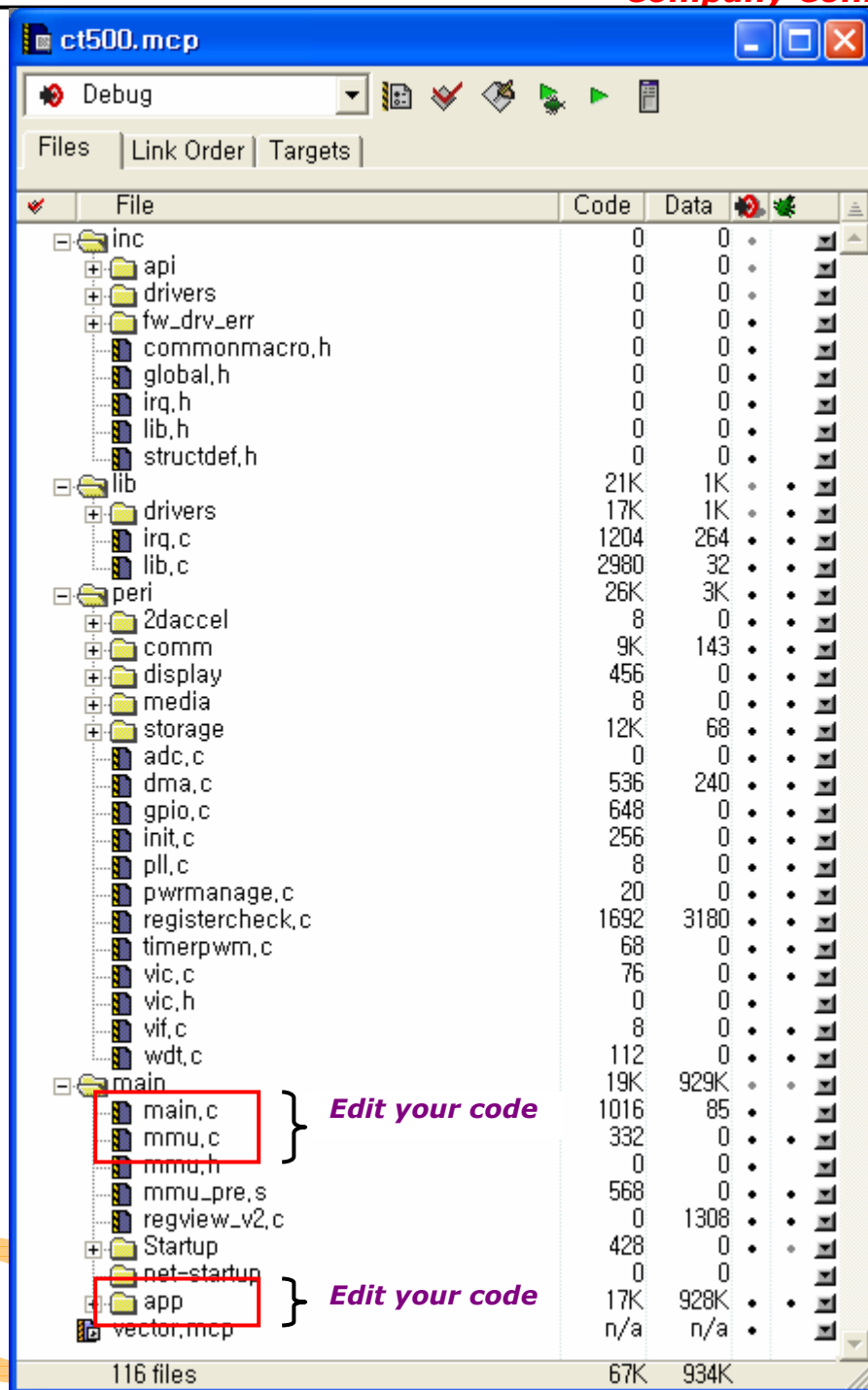
F/W 소스파일이 추가/삭제되었을 경우 추가/삭제된 파일에 대해 실행 파일 생성 시 반영이 되기 위해서는 makefile을 수정해 주어야 한다.

5.2. ADS v1.2 Using

5.2.1. FirmWare Project Structure

다음은 ADSv1.2 를 사용하여 구성한 CT-500 의 firmware project structure이다.

Confidential



[Fig - 6] ADSv1.2 Project Structure

6. HOWTO Porting

상화에서 제공되는 CT-500 용 F/W 소스를 해당 어플리케이션에 맞게 포팅하기 위해서는 다음과 같이 하면 된다.

- 1) 개발하고자 하는 플랫폼의 메모리 맵에 맞게 시스템 설정을 한다.

`syscfg.h`

- 2) 메모리 맵과 스택 영역에 맞추어 `startup` 코드를 수정한다.

`main/startup/startup.s`

- 3) 개발 어플리케이션에 맞게 소스를 작성한다.

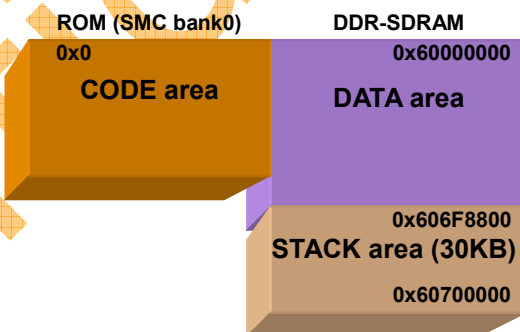
`main/mmu.c`

`main/app`

6.1. Memory map

상화에서 제공되는 F/W 소스의 메모리 맵을 다음과 같다.

- ROM (SMC bank0) base F/W 코드 영역
- DDR-SDRAM base data 영역
- DDR-SDRAM base stack 영역



[Fig - 7] Memory map

6.2. System setting

개발하고자 하는 제품에 맞도록 `syscfg.h`의 시스템 설정을 수정하면 된다. 그러나 특별한 경우를 제외하고 대개의 경우 상화에서 제공하는 플랫폼의 시스템 설정을 그대로 사용하면 된다.

6.3. Source modification

우선 메모리 맵과 스택 영역에 맞추어 `startup` 코드를 수정한다. 그 후 `mmu`와 어플리케이션에 맞게 `main/app` 디렉토리에 소스를 작성하면된다.

APPENDIX

A. ASSEMBLER DIRECTIVES

1) Unary Operators

- DEF - Usage -> :DEF:A {TRUE} if A is defined, otherwise {FALSE}
- NOT - Usage -> :NOT:A Bitwise complement of A
- LNOT - Usage -> :LNOT:A Logical complement of A

2) Logical operators

- OR - A:OR:B Bitwise OR of A and B
- EOR - A:EOR:B Bitwise Exclusive OR of A and B

3) Symbol definition directives

- GBLA - Declares a global arithmetic variable
- GBLL - Declares a global logical variable
- GBLS - Declares a global string variable
- SETA - Sets the value of a local or global arithmetic variable
- SETL - Sets the value of a local or global logical variable
- SETS - Sets the value of a local or global string variable

4) Data definition directives

- MAP - Sets the origin of a storage map to a specified address
 - MAP expr(,base-register} Address is expr + base-register's value
 - ^ is a synonym for MAP
- FIELD - Describes space within a storage map that has been defined using the MAP directive.
 - # is a synonym for FIELD
- LTORG - Literal pool(상수 공간)의 .org를 지정
- SPACE - Reserves a zeroed block memory
- DCD - Allocates one or more words(4Bytes) of memory
 - Defines the initial runtime contents of the memory

5) Miscellaneous directives

CODE16/CODE32

- Instructs the assembler to interpret subsequent instructions as 16/32bit instruction

6) Reporting directives

ASSERT- Generates an error message if a given assertion is false

OPT - Listing options

- 1: Turn on normal listing 2: off normal listing

- 4: Page throw 128: Turn off listing of macro expansions

Confidential

12	I	I-Cache	0=disable, 1=enable
13	V	High vector table	0=0x000000 에 vector table 위치 1=0xffff0000 에 vector table 위치

[Table - 3] MMU 제어 컨트롤 레지스터 cp15:c1 의 비트필드

프로세스 ID	Should be zero
31	0

[Fig - 10] 고속 문맥 전환 레지스터 cp15 레지스터 13

TTB(Translation Table Base)	Should be zero
31	0

[Fig - 11] 변환 테이블 베이스 주소 cp15 레지스터

ARM920T, ARM926EJ-S cache의 경우 I/D cache로 구성되며, 세가지 정책(write policy, cache replacement, cache allocate policy)에 있어 동일하다.

1) 다음은 I/D cache의 flush/clean에 사용되는 cp15 의 register 및 명령어이다.

기능	MCR 명령어	Remark
캐시 플러시	mcr p15, 0, Rd, c7, c7,0	ARM920T와 ARM926EJ-S 동일
데이터 캐시 플러시	mcr p15, 0, Rd, c7, c6,0	ARM920T와 ARM926EJ-S 동일
명령어 캐시 플러시	mcr p15, 0, Rd, c7, c5,0	ARM920T와 ARM926EJ-S 동일

[Table - 4] Cache flush 명령어

2) 다음은 way와 세트 인덱스 주소 지정 방식으로 cp15:c7 를 이용하여 D cache를 clean하는 명령어이다.

기능	MCR 명령어	Remark
명령어 캐시 라인 플러시	mcr p15, 0, Rd, c7, c5, 2	ARM926EJ-S(920T은 지원 안함)
데이터 캐시 라인 플러시	mcr p15, 0, Rd, c7, c6, 2	ARM926EJ-S(920T은 지원 안함)
데이터 캐시 라인 클린	mcr p15, 0, Rd, c7, c10, 2	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 클린 및 플러시	mcr p15, 0, Rd, c7, c14, 2	ARM920T와 ARM926EJ-S 동일

[Table - 5] Way 와 세트 인덱스 주소 지정 방식을 사용한 cache line clean & flush cp15:c7 명령어

ARM920T

31	26 25	8 7	5 4	0
Way	SBZ	Set	SBZ	

ARM926EJ-S

31 30 29	y x	5 4	0
Way	SBZ	Set	SBZ

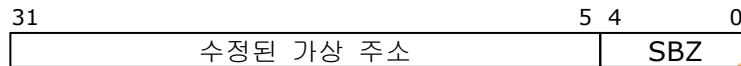
[Fig - 12] Way 와 세트 인덱스 주소 지정 방식을 사용한 cache clean 시 cp15:c7 레지스터 Rd 의 형식

3) 다음은 Cache의 일부분을 clean하고 flush하는 명령어이다. (가상 주소가 사용된다.)

기능	MCR 명령어	Remark
명령어 캐시 라인 플러시	mcr p15, 0, Rd, c7, c5, 1	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 플러시	mcr p15, 0, Rd, c7, c6, 1	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 클린	mcr p15, 0, Rd, c7, c10, 1	ARM920T와 ARM926EJ-S 동일
데이터 캐시 라인 클린 및 플러시	mcr p15, 0, Rd, c7, c14, 1	ARM920T와 ARM926EJ-S 동일

[Table - 6] 주메모리 안에서 위치를 참조하여 cache line clean 하고 flush 하는 명령어

ARM920T, ARM926EJ-S



[Fig - 13] 주메모리 안에서 원래 값으로 참조된 cache line 을 clean 하고 flush 할 때의 cp15:c7 레지스터 포맷

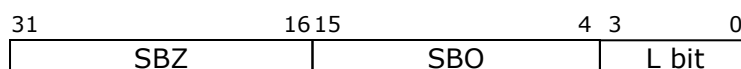
4) 락다운 된 cache 데이터/코드는 교체되지 않으며, cache가 flush되면 락다운 된 정보는 사라진다. Cache lock down 방법은 way addressing, lock down bit, 특수 allocate 명령어를 통한 방법이 있다. 특수 allocate 명령어를 통한 lock down은 xScale ARM을 위한 방법이며, way addressing 방법은 ARM920T와 ARM926EJ-S 동일하게 적용 가능하다. 반면 lock down bit 방법은 ARM926EJ-S(ARM920T 지원 안함)에 적용 가능한 방법이다.

ARM926EJ-S의 경우 way addressing과 lock down bit를 이용한 방법이 가능하나 여기서는 lock down bit 방법만을 기술하도록 한다.

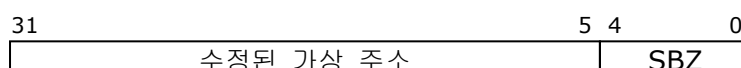
명령	MCR과 MRC 명령어
I 캐시 락 페이지 레지스터 읽기	mrc p15, 0, c9f, c9, c0, 1
D 캐시 락 페이지 레지스터 일기	mrc p15, 0, c9f, c9, c0, 0
I 캐시 락 페이지 레지스터 쓰기	mcr p15, 0, c9f, c9, c0, 1
I 캐리 락 페이지 레지스터 쓰기	mcr p15, 0, c9f, c9, c0, 0
주소에서 코드 캐시 라인 읽기	mcr p15, 0, Rd, c7, c13, 1

[Table - 7] Lock down bit 를 사용하여 cache 를 lock 하기 위한 cp15:c9 명령어

ARM926EJ-S



ARM920T, ARM926EJ-S



[Fig - 14] Lock down bit 사용 cache lock 했을 때 cp15:c9 레지스터 포맷

[Fig-9]의 첫번째 그림이 I/D cache lock bit와 관련 된 것이며, 두 번째 그림이 가상 주소를 이용한 cache line lock 방법 시 레지스터 포맷이다.

Confidential